OFFLINE CODING COMPETITION HANDBOOK

Project Case Study: Warkop Digital (Point of Sale System)
Duration: 180 Minutes | Language: HTML, Bootstrap 5, JavaScript (Vanilla)

1. Competition Overview

Welcome to the Offline Coding Competition. In this challenge, you are tasked with building a responsive, fully functional front-end **Point of Sale (POS) Application** for a local coffee shop called 'Warkop Terbuka'. You must build the entire system without internet access, using strictly standard vanilla JavaScript DOM manipulation, local mock databases (arrays), and Bootstrap 5 styles.

2. Core Architecture & Requirements

Your application must be built using a single-page structure separated into two distinct dynamic views controlled by JavaScript state: the **Main Ordering View** and the **Receipt View**.

2.1 Main Ordering View Requirements

• **Menu List Layout:** Display five initial database items dynamically: Black Coffee (Rp5,000), Milk Coffee (Rp7,000), Sweet Tea (Rp4,500), Boiled Noodles (Rp10,000), and Toast (Rp12,000).
• **Interactive Cart:** Implement a section displaying currently selected items, item prices, individual delete buttons (X), and a dynamic subtotal calculation section.
• **State Syncing:** Clicking the 'Add to Cart' button must immediately update the cart UI and re-tally the total price.

2.2 Receipt & Invoice View Requirements

• **View Switching:** Clicking 'Pay Now' must toggle class lists to completely hide the ordering screen and display the receipt card panel.
• **Empty Cart Check:** Implement strict validation preventing transitions if the cart array has zero items.
• **Invoice Tables:** Render the final ordered items using a standard borderless layout structure alongside a definitive total payment line.

3. Specific Implementation Tasks

Task 1: Bootstrap UI Setup & Dynamic View Layout

Build the primary interface container. Integrate Bootstrap 5 elements (cards, grids, list groups) and create two major component sections: and . The ordering section must feature a 2-column layout (MD-6 each) separating the menu deck from the active shopping cart layout.

Task 2: Dynamic JavaScript Menu Rendering

Define an array of strings representing the item names and a corresponding array of numbers representing the item prices. Write a function named renderMenu() that iterates through these database placeholders, maps them into structured HTML list nodes, and injects them directly into the DOM using an inner content setter.

```
Example Framework:
let daftarMenu = ['Kopi Hitam', 'Kopi Susu', ...];
let daftarHarga = [5000, 7000, ...];
```

Task 3: Interactive Cart Management (Push & Splice)

Implement two crucial functions: tambahKeKeranjang(index) and hapusDariKeranjang(index). Instead of copying text names, your cart state tracking should strictly push and splice numeric array indexes to simplify data binding. Every data mutation must immediately trigger an update function to ensure state visibility syncing.

Technical rule: Use the standard array manipulation tool .push(index) to add data, and .splice(index, 1) to pop target indices dynamically.

Task 4: State Validation & Invoice Render

Develop the prosesCheckout() routine. You must execute an explicit guard check using the array's boundary property. If empty, display a native warning alert and halt execution. If validated, calculate the final accumulated billing sum, inject a dynamic detailed listing, and switch display class tokens.

Task 5: Complete Reset & New Session Management

Write the closing function pesananSelesai() triggered by the final action button. This function must completely empty the target cart tracking state, trigger a cart re-render to display the empty placeholder, trigger a native completion banner alert, and swap the display visibility layout properties back to the default layout page state.